

Function Overloading , default
argument and inline function

Function overloading

- Function overloading is a C++ programming feature that allows us to have more than one function having same name but different parameter list, when we say parameter list, it means the data type and sequence of the parameters, for example the parameters list of a function myfuncn(int a, float b) is (int, float) which is different from the function myfuncn(float a, int b) parameter list (float, int).
- Function overloading is a compile-time polymorphism.
- Function overloading is usually used to enhance the readability of the program. If you have to perform one single operation but with different number or types of arguments, then you can simply overload the function.

Different ways to Overload a Function

- By changing number of Arguments.
- By having different types of argument.
- **Function Overloading: Different Number of Arguments**
- In this type of function overloading we define two functions with same names but different number of parameters of the same type. For example, in the below mentioned program we have made two sum() functions to display sum of two and three integers.

Example of By changing number of Arguments.

```
#include<iostream>
using namespace std;
Void sum (int x, int y)
{
    cout << x+y<<endl;
}
// second overloaded defintion
Void sum(int x, int y, int z)
{
    cout << x+y+z<<endl;
}
int main()
{
    sum (10, 20);
    sum(10, 20, 30);
    return 0;
}
```

Output:

30

60

Function Overloading: Different Datatype of Arguments

- In this type of overloading we define two or more functions with same name and same number of parameters, but the type of parameter is different. For example, we have two sum() function, first one gets two integer arguments and second one gets two double arguments.

Example of By having different types of argument.

```
#include<iostream>
using namespace std;
void sum(int x, int y)
{
    cout<< x+y<<endl;
}
void sum(double x, double y)
{
    cout << x+y;
}

int main()
{
    sum (10,20);
    sum(10.5,20.9);
    return 0;
}
```

Output:
30
31.4

Default Argument

- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value.

Example

```
#include<iostream>
using namespace std;
void print(int a, int b = 1, int c = 2, int d = 3)
{
    cout << "a: " << a << endl;
    cout << "b: " << b << endl;
    cout << "c: " << c << endl;
    cout << "d: " << d << endl;
}
int main()
{
    int a = 12, b = 13, c = 14, d = 15;
    cout << "only argument a passed\n";
    print(a);
    cout << "only arguments a,b passed\n";
    print(a, b);
    cout << "only arguments a,b,c passed\n";
    print(a, b, c);
    cout << "All arguments a,b,c,d passed\n";
    print(a, b, c, d);

    return 0;
}
```

Output:

only argument a passed

a: 12

b: 1

c: 2

d: 3

only arguments a,b passed

a: 12

b: 13

c: 2

d: 3

only arguments a,b,c passed

a: 12

b: 13

c: 14

d: 3

All arguments a,b,c,d passed

a: 12

b: 13

c: 14

d: 15

Rules for default argument

- In the function definitions, if we declare default argument for one variable, then all the variables after that need to have default arguments.
 - `int func(int a, int b=0, int c) //error`
 - `int func(int a, int c ,int b=0) //correct`
- The default value for arguments are being copied if the argument value is provided while calling the function
- Values are assigned from left to right.

Inline Function

- Inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of inline function call. This substitution is performed by the C++ compiler at compile time. Inline function may increase efficiency if it is small.
- The syntax for defining the function inline is:

```
inline return-type function-name(parameters)  
{  
    // function code  
}
```

Inline Function

- Remember, inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining. Compiler may ignore the inline request :
 - If a function contains a loop. (for, while, do-while)
 - If a function contains static variables.
 - If a function is recursive.
 - If a function return type is other than void, and the return statement doesn't exist in function body.
 - If a function contains switch or goto statement.

Advantage of inline function

- Function call overhead doesn't occur.
- It also saves the overhead of push/pop variables on the stack when function is called.
- It also saves overhead of a return call from a function.
- Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return.

Example

```
#include <iostream>
using namespace std;
inline void cube(int s)
{
    cout<<s*s*s;
}
int main()
{
    cout << "The cube of 3 is: ";
    cube(3);
    return 0;
}
```

Output:
The cube of 3 is: 27